Belief–Effect Mismatch: Attributing Tool-Side and Belief-Side Failures in Tool-Using LLM Agents

Viet Phuong Nguyen

Placeholder Institute, Vietnam
24022431@vnu.edu.vn

Abstract. An LLM agent acting through tools can fail because a tool broke its own contract or because the agent formed a wrong belief about what it did, and the two look identical in the agent's final report—yet they demand opposite recoveries. We study whether the two causes can be told apart when the true cause is known. We build a controlled testbed over an MCP filesystem server in which every trial carries a ground-truth cause: clean, a tool-side world_drift injected by operators that silently violate a tool's unchanged description, or a belief-side halluc. Matched pairs hold the instruction and initial world fixed so the injected cause is the only difference, and an out-of-band oracle supplies a cause label the agent never sees. On this testbed we evaluate a training-free probe that attributes failure along two orthogonal binary axes—whether each tool honoured its description, and whether the agent's account survives contact with the true world—combined into a 2×2 decision whose fourth cell flags cases an agent should defer to a human. On the trials where a fault leaves any trace in the effect, the tool-side axis recovers the fault exactly (six of six visible drifts), with perfect precision on both fault classes; every error is a missed fault read as clean, never one cause confused for the other, and two attribution-free baselines recover none of these drifts by construction. We further isolate a class of *invisible drift* that no effect-based probe could attribute, and re-score under a post-hoc verified ground truth that replaces design-time labels with what actually occurred: on the observable, verified trials the probe attributes every one correctly, though the testbed yields a single genuine belief-error trial, so belief-side attribution is shown as an existence proof rather than a rate. Throughout, the probe is a deterministic oracle measuring what the two-axis formulation can recover from perfect evidence—an upper bound—not an agent introspecting on itself.

Keywords: failure attribution · tool-using agents · MCP · controlled evaluation.

1 Introduction

An LLM agent acting through tools can fail for two very different reasons that look identical from the outside. It may fail because a tool did not do what its description promised—a write that silently appended instead of replacing, a delete that quietly moved a file to a trash folder. Or it may fail because the agent itself formed a wrong belief about what it did—reporting a file moved when it was only copied, a count that was never checked, a write that never happened. In both cases the task is left incomplete and the agent's report may sound equally confident, yet the right response is opposite: a broken

tool contract calls for re-reading the world and replanning; a mistaken belief calls for the agent to re-examine its own reasoning. An agent that cannot tell these apart cannot choose the right recovery.

Current evaluation makes this hard to study. Tool-use and MCP benchmarks report whether a task succeeded, not why it failed, so a tool-side fault and a belief-side fault collapse into the same failed score. Recent diagnostic work localizes failures within a trajectory, but post hoc, over broad taxonomies, and against ground truth annotated by hand. None of these lets us ask the sharp question we care about: when a failure could have come from either the tool or the agent's belief, and the true cause is known, can it be attributed correctly?

To ask that question we need trials whose cause is fixed by construction. We build a small, controlled testbed over a single MCP filesystem server in which every trial carries a ground-truth cause—clean, a tool-side world_drift, or a belief-side halluc—established by how the trial is made rather than judged afterward. Tool-side faults are injected by drift operators that leave a tool's description untouched while silently changing its behaviour, modelling a phenomenon measured in roughly one in eight real MCP servers. Matched pairs hold the instruction and the initial world fixed across conditions, so that the injected cause is the only thing that differs and cannot be read off surface text. An out-of-band oracle observes the true world state, giving each trial a cause label the agent never sees.

On this testbed we test a training-free attribution probe that splits the diagnosis into two orthogonal binary axes. Axis A checks whether each tool honoured its description; a failure here is tool-side. Axis B checks whether the agent's own account survives contact with the true world; a failure here is belief-side. The two answers combine into a 2×2 decision whose fourth cell—both axes failing—names exactly the case where the agent should stop and ask a human rather than attempt a repair it cannot ground.

Our contributions are: (C1) A controlled testbed of 30 matched-pair trials over an MCP filesystem server, in which every failure carries a ground-truth cause label established by construction rather than annotation. **(C2)** Evidence that tool-side drift and belief-side hallucination are indistinguishable from the agent's final report alone, motivating attribution from observed effects. **(C3)** A two-axis attribution probe that, on trials where a fault leaves any trace in the effect, recovers the tool-side fault source exactly (all six visible drifts, F1=1.00) with perfect precision on both fault classes; two attribution-free baselines recover none by construction. **(C4)** Identification of a class of *invisible drift*—faults that leave no trace in the effect and are unattributable in principle, even to an oracle—isolated rather than left to dilute the reported metrics. **(C5)** A post-hoc verification protocol replacing each design-time label with the fault that actually occurred at run time; under it, the probe attributes every observable, verified trial correctly, while exposing that the testbed produced only one genuine belief-error trial.

We are explicit about scope. This is a first, controlled step: a single agent backbone, a single tool domain, thirty trials. We do not claim that the agent introspects on itself, nor that attribution improves recovery; the probe is a deterministic oracle that measures what the two-axis formulation can recover when the evidence is perfect. We report, as a finding rather than a footnote, that design-time halluc labels do not guarantee a hallucination occurs at run time, and we re-score against a verified ground truth to show what the

labels alone obscure. The testbed and probe are built to make these boundaries visible, not to hide them.

2 Related Work

Tool-use and MCP benchmarks. A large and fast-growing body of work evaluates how well agents use tools. Benchmarks such as τ-bench and its successors, along with MCP-specific suites like MCP-Atlas [1] and MCPAgentBench [2], measure whether an agent completes a task, often over realistic multi-step workflows and, in the MCP case, over real or simulated servers. These benchmarks are indispensable for ranking systems, but they share a structural limitation for our purposes: they report a single end-to-end success or failure signal and do not localize where in the agent's process a failure originates [3]. A task marked failed could have failed because a tool misbehaved or because the agent formed a wrong belief, and the benchmark score cannot tell the two apart. Our testbed is built precisely to make that distinction, and so is complementary to these suites rather than a competitor to them: it trades their scale and realism for controlled, cause-labelled trials.

Failure attribution and trajectory diagnosis. A more recent line of work moves from scoring outcomes to diagnosing failures. Closest to ours is AgentRx [4], which localizes a trajectory's critical failure step and classifies it under a nine-category, cross-domain taxonomy, using an LLM-based judge over an auditable log of constraint violations; related efforts diagnose failures in agentic coding systems [5] and survey the fragmented state of agent evaluation [6]. Our work shares the attribution goal but differs in three ways. AgentRx operates post hoc on trajectories already known to have failed; our probe runs online, predicting an effect before each action and checking it against the world immediately after. AgentRx's ground truth comes from manual annotation against a broad taxonomy; ours comes from controlled injection, so every trial's cause is fixed by construction rather than judged after the fact. And we deliberately restrict attribution to two orthogonal binary axes—each anchored to a concrete, checkable signal—rather than a multi-way taxonomy that requires a judge. The two designs answer different questions: what broad class of error occurred in a real failed run, versus whether two specific and easily-confused causes can be told apart when the ground truth is known.

Description-code inconsistency in MCP servers. Our tool-side fault model is not hypothetical. Two independent large-scale studies establish that MCP tools frequently diverge from their own descriptions. A static-analysis study of 10,240 real-world servers finds that roughly 13% exhibit substantial description–implementation mismatches [7], and a second measurement over 19,200 description–code pairs from 2,214 servers reports a comparable 9.93% inconsistency rate under a different detection method [8]. These studies frame the phenomenon primarily as a security surface—undocumented privileged operations and hidden side effects—and they measure its prevalence rather than an agent's ability to notice it at run time. We take the same phenomenon as the basis for our world_drift operators, and ask the complementary question they do not: when a tool

silently violates its description, can the resulting failure be attributed to the tool rather than to the agent?

Belief and confidence in agents. Our belief-side axis relates to work on whether agents represent their own uncertainty and know when to call a tool. We do not probe internal representations; our signal is behavioural, read from whether the agent's stated account survives contact with the true world. Making it introspective—eliciting a structured belief and confidence before acting—is future work.

3 Problem Formalization

We consider an agent carrying out a task by issuing a sequence of tool calls to an MCP server. At each step the agent holds a belief about what a call will do, issues the call, and observes a report returned by the tool. The call also produces a real change in the world—the filesystem state—which the report describes but does not constitute. A failure is a mismatch between what the task required and what the world ultimately contains. Our question is not whether such a mismatch occurred, but what produced it.

Two sources of failure. We separate the causes of a step-level failure into two sources that are, in principle, independent.

The first is *tool-side*: the tool's behaviour diverges from its own description. The description is a contract—this call replaces the file, this call permanently deletes—and the tool violates it silently, doing something the description does not license. This source is aleatoric from the agent's perspective: nothing in the information available to the agent before the call reveals the divergence, because the description, which is all the agent has, is exactly what the tool is failing to honour. We label a failure from this source world_drift.

The second is *belief-side*: the agent's belief about the effect of its own actions is wrong, even though every tool behaved as described. The agent may believe it wrote a file it never wrote, moved a file it only copied, or produced a result it never produced. This source is epistemic: it lies in the agent's model of the world, not in the world's behaviour, and better reasoning over the same evidence could in principle have avoided it. We label a failure from this source halluc.

Attribution as a two-axis decision. Because the two sources are independent, a principled attribution does not choose one label from a flat list; it answers two separate yes/no questions. Axis A asks whether each tool honoured its description. Axis B asks whether the agent's account of what it achieved survives comparison with the true world. Writing A for "the tool matched its description" and B for "the belief matched the world," the two answers define four outcomes: (A, B) clean; $(\neg A, B)$ tool-side, world_drift; $(A, \neg B)$ belief-side, halluc; and $(\neg A, \neg B)$ *both*, where tool and belief have each failed. The final cell is not redundant: it names the situation in which the agent cannot safely repair on its own, and the appropriate response is to stop and defer to a human rather than act on a compromised understanding. A scheme that collapsed attribution into a single label could not express this cell, which is the central reason we keep the axes separate.

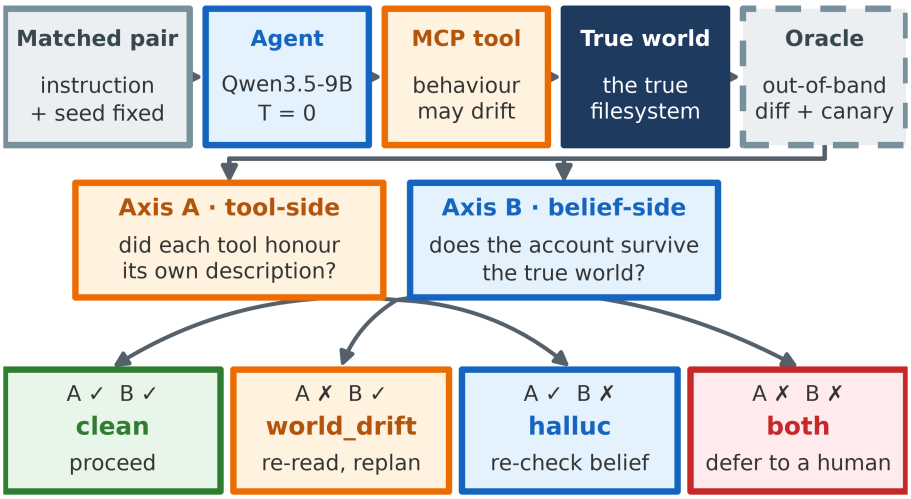

Fig. 1. Execution flow of a single trial: a matched-pair task runs through the agent and the (possibly drifted) tools into the true world, which an out-of-band oracle reads to drive the two-axis attribution into a 2×2 decision. The agent sees only descriptions and reports; ground truth lives entirely in the oracle.

4 Testbed Design

Existing agent benchmarks record whether a task succeeded, but not why it failed. That is enough to rank systems and insufficient to study attribution: to ask whether a divergent tool can be separated from the agent's own mistaken belief, we need trials whose cause is fixed by construction rather than guessed after the fact. Our testbed supplies this. It contains 30 trials over a single Model Context Protocol (MCP) filesystem server, each carrying a ground-truth cause label—8 clean, 14 world_drift, and 8 halluc. Fig. 1 shows the end-to-end flow of a single trial, from a matched-pair task through the agent and tools into the true world, which an out-of-band oracle reads to drive the two-axis attribution.

The server exposes six tools: `write_file`, `append_file`, `read_file`, `list_dir`, `delete_file`, and `acquire_lock`. The same six tools, carrying the same natural-language descriptions, are present in every condition. Nothing the agent can observe about the interface distinguishes a clean trial from a faulted one—a property we rely on to keep each injected cause hidden from the agent under test.

4.1 Construction Methodology

The testbed follows a deliberate construction process: producing trials whose cause is known with certainty while remaining, from the agent's point of view, indistinguishable from an ordinary tool-use task. We describe that process here because how a testbed is built bounds how much its results can be trusted.

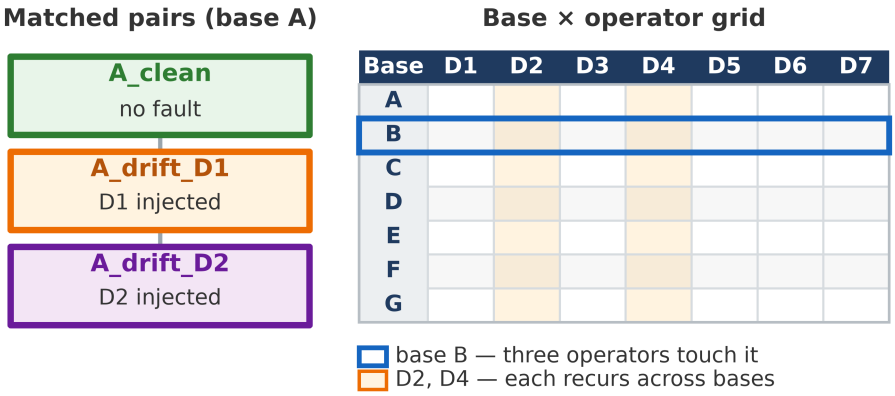

Fig. 2. Matched-pairs construction (left) and the base–operator grid (right): a matched-pairs group shares an instruction and seed so only the injected fault differs (A_drift_D2 is simply a second world_drift example, not a *both* case), and no base implies a fixed operator.

A documented, unattributed phenomenon. Description–code inconsistency in MCP servers is real and measured at scale: a static-analysis study of 10,240 servers reports ≈13% substantial mismatches [7], and an independent measurement over 19,200 pairs from 2,214 servers reports 9.93% under a different detection method [8]. No existing benchmark isolates this tool-side cause from agent-side belief error as a distinct, attributable category. The testbed exists to close that gap, not to be a general-purpose evaluation.

Base selection and operator design. We started from seven short filesystem workflows and asked, for each, where a tool could plausibly diverge from its description without being contrived. A workflow qualifies as a base only if some step has a documented contract specific enough to be silently violated. Each of the seven operators (Table 1) was designed against two constraints: (i) leave the tool's description exactly as written; (ii) alter exactly one aspect of behaviour, so a failure traces to a single nameable cause. Each operator also has a halluc counterpart on the same base, so drift and halluc trials share surface characteristics—length, tool sequence, phrasing—and the two causes cannot be told apart by anything other than the effect itself.

Verification, and an error the process caught. The canary mechanism and oracle inspector, which carry the full weight of ground truth, were exercised with synthetic hand-constructed world states before any live trial: the canary check against present-at-expected-path, absent, and present-at-wrong-path cases; the content-level classifier against synthetic before/after pairs triggering every category (replaced, appended, prepended, truncated, wrapped, filled from empty, emptied)—the distinction that separates a D1 drift from an ordinary overwrite. Cross-checking every task's declared operator against the server's implemented operators caught, before any GPU time, that two operators (D6, stale cached read; D7, duplicated write) were initially specified in tasks but not yet implemented, so their trials would have silently run as clean. We report this because a construction methodology whose failure modes are never visible is not one we would trust either.

Table 1. The seven drift operators. In every case the tool's description is left unchanged; only its behaviour diverges from that description.

Operator	Divergence from the tool's own (unchanged) description
D1	appends instead of replacing
D2	imposes an undocumented lock precondition
D3	alters the format of a returned listing
D4	silently relocates a write into a hidden namespace
D5	soft-deletes to a trash path instead of removing
D6	serves a stale cached copy on read
D7	applies an append twice

4.2 Tasks, Matched Pairs, and Drift Operators

The 30 tasks are built from seven bases: A overwrites a file, B deletes a file and counts what remains, C creates a file under a specific directory, D writes under a precondition the documentation does not mention, E lists a directory, F follows a configuration file to a target and writes there, and G appends to a log with existing content. Bases differ in what makes them fragile—A depends on replacement semantics, C on where a write lands, F on reading the right target first—so different drift operators have somewhere to bite. Each base appears under several conditions sharing the same instruction and seed filesystem; only the injected fault differs. We avoid a one-to-one base–operator mapping: base B is exercised by three operators and single operators recur across bases, so a model cannot fall back on a "base B means operator X" shortcut.

A world_drift is created by a drift operator: a tool whose description is untouched but whose behaviour quietly departs from it. The seven operators (Table 1) span the ways a tool's behaviour can silently diverge from a fixed description: appending where the description promises replacement (D1); imposing an undocumented precondition such as locking (D2); altering a return format (D3); relocating a write into a hidden namespace, so `notes/summary.txt` actually lands at `drafts/notes/summary.txt` (D4); soft-deleting to a trash path instead of removing (D5); serving stale cached reads (D6); and applying an append twice (D7).

4.3 Canary Tokens and the Oracle Inspector

A fluent model can describe an action convincingly enough to pass a shallow check without the action having occurred. To close this gap, every task whose workflow includes a write step—bases A, C, F, and G—embeds a fixed six-hex canary token in the content the agent must write. A task might ask the agent to write `"Q3 revenue: 160 [2d232f]"`, where `2d232f` is the canary. Because the token is unguessable, the true world state contains it only if the agent actually performed the write; success claims made without the canary landing are, by construction, belief failures with no need to parse free-form text. The number of occurrences carries further information: a task writes its canary exactly once, so a count above one means the write was multiplied or the original left in place—which is how a relocation the agent only believes it performed

(copy rather than move) is caught. Canaries live only in content the agent must produce, never in seeds; they are fixed per base so matched pairs share a token and runs reproduce. Read- and count-oriented bases (B, E) carry no canary; there, a wrong count is itself the belief-side signal.

Ground truth is delivered by an out-of-band inspector that observes the true filesystem directly and is never exposed to the agent. It snapshots the world before and after each step and computes a two-tier diff: path-level (which files were created, deleted, modified) and content-level (how each modified file changed: replaced, appended, prepended, truncated, wrapped, filled from empty, emptied). The second tier is what makes drift detectable at all: a hash-only diff would collapse an append and an overwrite into one "modified" signal, which is precisely the difference between a D1 drift and a correct replacing write. Alongside the diff, the inspector reports whether a given canary reached the world, at which paths, and how many times—separating a namespace drift (wrong location) from a missing write (belief failure) by location alone.

5 The Two-Axis Attribution Probe

Given a trajectory and the world state it produced, the probe must decide whether a failure came from the tool or from the agent's belief. Rather than ask that question directly—which forces a single fuzzy judgement over a continuum of failure modes—we split it into two checks that can each be answered from concrete evidence, and combine their outcomes.

Axis A asks whether each tool did what its own description says. This is tool-side, and a failure on this axis corresponds to world_drift. Axis B asks whether the agent's own account survives contact with the true world. This is belief-side, and a failure here corresponds to halluc. The axes are independent: a trial can fail neither, either, or both.

In this work each axis is realised as a deterministic rule over the oracle's evidence—the observed effect, the tool descriptions, and the canary report. We use a rule-based oracle rather than a model-based judge on purpose: it establishes what the two-axis formulation can recover when the evidence is perfect, before any question of whether a model can approximate that judgement.

Axis A: does the tool match its description? Axis A walks the trajectory step by step and checks the observed effect of each tool call against a small contract derived from that tool's description. For a write, the description promises replacement, so the content-level change must be "replaced" or "filled from empty"; an "appended" change signals a D1 drift, unless the agent's own submitted content already contained the prior text (in which case the agent composed the combined text itself and the tool is not at fault). For an append, the permitted change is "appended". A write whose created path differs from the path the agent requested is a relocation, flagging D4. For a deletion, the description promises permanent removal, so a delete that creates a "leftover" copy in a trash path signals a D5 soft-delete. Two further signals are checked before the per-step walk: if the canary appears in the world more than once, a single write was multiplied, flagging D7; and if the agent never terminated—exhausting its step budget while a tool repeatedly returned an error and the world never changed—the tool blocked progress, the signature of a D2 lock. In all cases axis A fails, and the trial is attributed tool-side.

Axis B: does the belief survive the world? Axis B compares the agent's closing account against the true final world. It relies on the canary as hard evidence wherever a task carries one. If the agent claims success but the canary never reached the world, the intended write did not land and the belief has failed against reality. If a task requires moving a file—so its canary begins in a seed and should end in exactly one place—and the canary instead survives in more than one location, the agent copied rather than moved and only believes it relocated the file. Absent a canary, the probe falls back to a narrow check: a success claim over a world that shows no change at all is treated as a belief failure. These checks are deliberately conservative, so that a flag on axis B is reliable even though many belief errors will pass unflagged.

Combining the axes. The two outcomes map onto a 2×2 table. When both axes pass, the trial is clean. When axis A fails and axis B passes, the fault is tool-side: world_drift. When axis A passes and axis B fails, it is belief-side: halluc. When both fail, the evidence implicates tool and belief at once; the agent cannot safely repair on its own, and the appropriate action is to halt and defer to a human rather than guess. This last cell is the reason for keeping the axes separate: a scheme that emitted a single label could not express "both, so stop."

6 Results

We run all 30 trials through a single agent backbone (Qwen3.5-9B, temperature 0, reasoning disabled) and pass each resulting trajectory to the oracle attribution probe of Section 5. On every trial the probe sees the true observed effect, the tool descriptions, and the canary evidence, but never the ground-truth label. It returns one of three decisions—clean, world_drift, or halluc—which we compare against the label. We report results in four passes: against the design-time labels over all 30 trials (Section 6.1); against two attribution-free baselines (Section 6.2); after separating the drifts that leave a trace from those that cannot (Sections 6.3–6.4); and finally against a post-hoc verified ground truth that replaces each design-time label with what actually occurred at run time (Section 6.5).

6.1 Overall Attribution

Table 2 reports per-class precision, recall, and F1 across all 30 trials; overall accuracy is 0.50 (15 of 30 trials attributed correctly). We read this table as three separate claims rather than a single headline number.

Primary result — observable tool-side faults are recovered. Every fault the probe commits to is correct: both fault classes reach a precision of 1.00. On the drift trials whose fault reaches the world (Section 6.3), the tool-side axis recovers the source exactly. The probe is conservative—it names a fault source only when the evidence forces it.

Secondary diagnostic property — the two axes never cross-confuse. The confusion matrix (Table 3) shows that every error lies in the clean column: a missed fault is always read as clean, never as the wrong kind of fault. No drift is ever attributed to belief, and no belief error to the tool. When the probe fails, it fails by staying silent, not by pointing at the wrong source—the property the two-axis decomposition was designed to give.

Table 2. Per-class attribution over all 30 trials. Both fault classes reach perfect precision; the recall gap is the subject of Sections 6.3–6.5. Overall accuracy is 0.50.

Class	n	Precision	Recall	F1
clean	8	0.35	1.00	0.52
world_drift	14	1.00	0.43	0.60
halluc	8	1.00	0.12	0.22

Table 3. Confusion matrix. Rows are ground truth, columns are the probe's prediction. The off-diagonal mass sits entirely in the clean column: the probe under-detects but never cross-confuses the two fault sources.

Ground truth ↓ / prediction →	clean	world_drift	halluc
clean	**8**	0	0
world_drift	8	**6**	0
halluc	7	0	**1**

Limitation — conservatism costs recall. That same conservatism suppresses recall, 0.43 for drift and 0.12 for hallucination over the design-time labels. The rest of this section accounts for that silence, and shows that most of it is a property of the trials rather than a shortfall of the probe.

6.2 Comparison to Attribution-Free Baselines

Before analysing the recall gap, we ask whether the two-axis probe earns its complexity: does a simpler rule recover the same tool-side faults? Two attribution-free baselines answer this, and their behaviour is fixed by the construction of the testbed rather than measured empirically. An *outcome-only* baseline sees only whether the task ultimately succeeded or failed; it has no channel on which to separate a tool fault from a belief error, so on the fault-source distinction it recovers nothing—zero of the six visible drifts. A *tool-report-only* baseline attributes the fault using only what each tool returned to the agent; because a drift operator is defined as a tool that reports success while silently violating its description, this baseline reads every drift as a clean success, again recovering zero of six. The two-axis probe, which reads the true observed effect rather than the report, recovers all six (Table 4). The gap is not incidental—it is the whole point of attributing from effect rather than from outcome or report. We state these two baselines analytically because the construction forces their result; an empirical baseline that a model must approximate—an LLM judge over the trajectory—is the comparison we leave to future work (Section 7).

6.3 Visible and Invisible Drift

A drift can be attributed from an effect only if it leaves a trace in that effect. Of the 14 world_drift trials, eight do not, for three reasons. Two trials on base B (B_drift_D1,

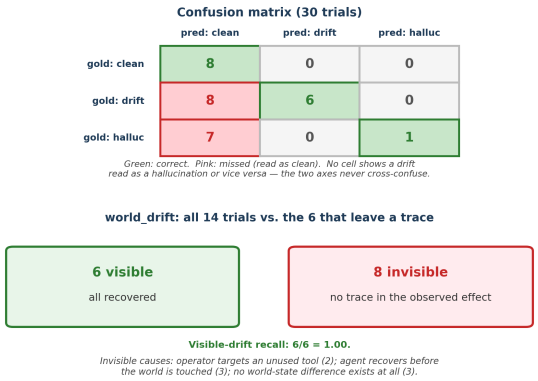

Fig. 3. Confusion matrix over all 30 trials, and the visible/invisible split of the 14 world_drift trials.

Table 4. Recovery of the six visible tool-side drifts. Both attribution-free baselines miss all six by construction; only a probe that reads the true effect recovers them.

Method	Recovers visible drift	Basis
outcome-only	0 / 6	forced by construction
tool-report-only	0 / 6	forced by construction
two-axis probe (ours)	**6 / 6**	empirical

B_drift_D4) apply a write-corrupting operator to a workflow that only deletes, so nothing changes. Three D2 trials (A_drift_D2, C_drift_D2, D_drift_D2) have the agent recover from the undocumented lock and retry, so the final filesystem is identical to a clean run. And three trials touch only tool internals: E_drift_D3 alters a return format only; F_drift_D6 and G_drift_D6 serve stale reads whose following writes are themselves well-formed, so only the choice of target is wrong, not the write. These eight are unattributable in principle—no probe reading the effect could recover them—and are reported separately rather than counted as errors.

Setting these eight aside isolates the cases where attribution is even possible. On the remaining 22-trial visible subset—all clean and hallucination trials, plus the six drifts that do leave a trace—the picture changes sharply: tool-side attribution becomes exact (drift P=R=F1=1.00, all six visible drifts recovered), and overall accuracy rises from 0.50 to 0.68; the design-time-visible row of Table 5 records this. The two-axis mechanism recovers the fault source whenever the fault is observable at all; the earlier recall gap for drift was almost entirely the invisible trials, not the probe.

6.4 Belief-Side Attribution

Hallucination recall stays at 0.12 even on the visible subset, and the reason is worth stating carefully, because it is not what the number first suggests. We inspected all eight halluc trials individually against the true final world. In six of them the agent's closing report matched what had actually happened. B_halluc2 is representative: the task was

built to tempt the agent into overlooking files inside a subfolder and reporting too low a count; instead the agent listed the subfolder, counted four files, and wrote four. A seventh trial, B_halluc, is ambiguous rather than clearly negative—the seed includes a hidden dotfile, the agent reports only the visible files, and whether that counts as an error depends on whether "files" was meant to include hidden ones, which the task does not say. The trial is labelled halluc because the task was designed to invite a belief error, but at temperature 0 this capable backbone simply did not commit one.

One trial, C_halluc, does contain a genuine belief error, and the probe catches it. The task asks the agent to move a file so that it lives under `notes/`. The agent instead copies the file, leaves the original in place, and reports that the relocation is complete. Because the canary now appears in two locations rather than one, the probe reads the duplication as a belief-side failure: the agent believes it moved a file it in fact only copied. This is the single case the probe flags, and it flags no clean trial as a hallucination, which is why belief-side precision is 1.00.

The low recall, then, reflects the trials more than the probe: seven of the eight halluc trials contained no belief error for any method to detect. The one that did was detected. This does, however, point to a real weakness in how the testbed assigns ground truth, which we address in Section 6.5 by re-scoring against what actually occurred.

6.5 Post-hoc Verified Ground Truth

Section 6.4 showed that seven of the eight halluc-designed trials contained no belief error to detect: the design-time label marked a task built to invite a hallucination, not one in which a hallucination occurred. Scoring against those labels therefore penalises the probe for correctly staying silent. We re-score under a verified ground truth that replaces each design-time label with what actually happened at run time, following the protocol in Fig. 4: designed fault → agent execution → actual trajectory and final world → post-hoc verification → actual label → attribution.

The verification touches only the eight halluc-designed trials—clean and drift labels are fixed by injection, not by agent behaviour. Of the eight, six show no belief error and are relabelled clean, one (B_halluc) is ambiguous and set aside, and one (C_halluc) is a genuine belief error. Table 5 compares attribution under the two label sets. Under verified labels the overall accuracy over the 29 scored trials rises to 0.72, and on the visible subset—every trial whose fault can leave a trace, with verified ground truth—the probe attributes all 21 correctly: 14 clean, 6 drift, and the single genuine hallucination.

Two caveats bound this result. First, the visible subset still excludes the eight invisible drifts, which no effect-based probe can recover. Second, and more important, the verified halluc class contains exactly one trial: belief-side attribution is demonstrated here as an existence proof—the one genuine belief error is caught, with no false positive on any clean trial—not as a rate. Establishing a belief-side recall worth reporting requires a testbed that reliably induces belief errors—many runs, higher temperature, or weaker backbones—which we identify as the priority next experiment.

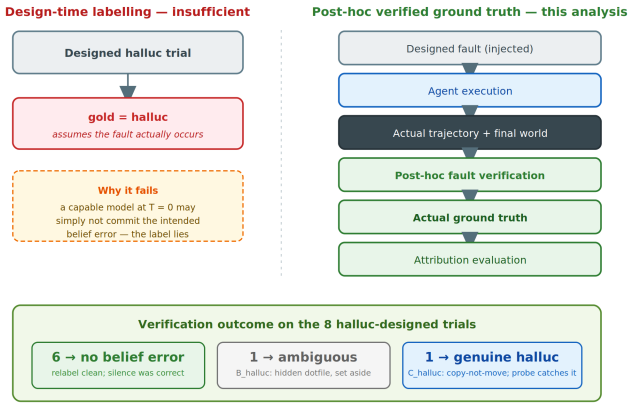

Fig. 4. From a design-time label to a post-hoc verified ground truth, and the verification outcome on the eight halluc-designed trials: six show no belief error, one is ambiguous, one is a genuine hallucination the probe catches.

Table 5. Attribution under design-time versus post-hoc verified labels. Verifying what actually occurred removes the penalty for correctly-silent halluc trials; on observable, verified trials the probe is exact. The belief column denotes genuine belief errors caught over genuine belief errors present.

Label set	Scored n	Accuracy	Drift F1	Belief evidence
design-time, all	30	0.50	0.60	1 caught / 8 labelled
design-time, visible	22	0.68	1.00	1 caught / 8 labelled
verified, all	29	0.72	0.60	1 / 1 genuine
verified, visible	**21**	**1.00**	**1.00**	**1 / 1 genuine**

7 Limitations

Belief-side ground truth rests on a single trial. We now report both design-time and post-hoc verified labels (Section 6.5), which removes the earlier objection that attribution was measured against labels that need not hold at run time. The residual limitation is sharper: with this backbone at temperature 0, only one of eight halluc-designed trials produced a genuine belief error, so belief-side attribution is an existence proof, not a rate. A sound belief-side evaluation needs a testbed that reliably induces belief errors—many runs, higher-temperature sampling, or weaker backbones.

The oracle, and the baselines, are rule-based. Both axes are deterministic checks over perfect evidence—intentional, since it measures what the two-axis formulation can recover in principle, but it means axis B only catches errors reducible to a canary or a no-change signal. Belief errors that require reading the semantics of a report—an agent that miscounts while the canary is present and correct—are outside its reach. The two baselines of Section 6.2 are likewise analytical; extending axis B with a model-based judge and benchmarking it against an LLM-judge baseline is a natural next step.

A single backbone and domain. All trials run one model on one filesystem server; the invisible-drift analysis in particular is entangled with agent behaviour (three D2 trials are unattributable precisely because this agent recovered from the lock, and a weaker agent might leave a detectable trace). Results across several backbones, and a second tool domain with numeric rather than filesystem effects, are needed before the visibility boundary can be claimed as a property of the operators rather than of one agent.

Attribution is not yet repair. We show that fault sources can be recovered when they are observable; we do not yet show that recovering them leads to better recovery than retrying blindly, an experiment we leave to future work.

8 Conclusion

We asked whether a divergent tool can be told apart from an agent's own mistaken belief when the true cause of a failure is known, and built a controlled testbed and two-axis probe to answer it. When a fault leaves any trace in the observed effect the tool-side axis recovers it exactly, both fault classes are attributed with perfect precision, and two attribution-free baselines recover none by construction; what the probe misses, it misses by staying silent rather than by confusing one cause for the other. Much of that silence is a property of the trials—eight of fourteen drifts leave no trace, and six of eight hallucination-designed tasks provoked no hallucination in a capable model at temperature zero; a post-hoc verified ground truth confirms the point on the observable trials and shows that the testbed yielded only one genuine belief error. Two directions follow: extend axis B with a model-based judge and benchmark it against an LLM-judge baseline, quantifying how much attribution survives without perfect evidence; and show that a recovered fault source leads to better recovery than retrying blindly—and that an agent knows when to stop and defer to a human. The present work is the controlled first step those experiments need.

References

1. Bandi, C., et al.: MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers. arXiv:2602.00933 (2026)
2. Liu, W., et al.: MCPAgentBench: A Real-world Task Benchmark for Evaluating LLM Agent MCP Tool Use. arXiv:2512.24565 (2026)
3. Evaluating Tool-Using Language Agents: Judge Reliability, Propagation Cascades, and Runtime Mitigation in AgentProp-Bench. arXiv:2604.16706 (2026)
4. Barke, S., Goyal, A., Khare, A., Singh, A., Nath, S., Bansal, C.: AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475 (2026)
5. Wang, M., Xie, X., Huo, Y.: TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563 (2026)
6. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv:2605.20530 (2026)
7. Li, Z., et al.: Don't Believe Everything You Read: Understanding and Measuring MCP Behavior under Misleading Tool Descriptions. arXiv:2602.03580 (2026)
8. Shi, Y., et al.: Description-Code Inconsistency in Real-world MCP Servers: Measurement, Detection, and Security Implications. arXiv:2606.04769 (2026)